

petitor in the space as Intel has already outlined its plans for 2020 but for now, let's take a look at what we have.

THE GRAPHICS PIPELINE

We experience everything on the humble display screen, from rich pictures which are 2-dimensional to even more engaging 3-dimensional content such as video games. Even has content has evolved and we have 3D content, at the end of the day, we still experience it on the flat 2-dimensional display. So a lot of the calculations involved in GPUs are about converting content from any format onto the 2D screen. Both AMD and NVIDIA have their unique ways of doing this with their proprietary APIs which run very fast on their own graphics accelerators. Or one can use an open source standard such as OpenGL which were created to ensure that programmers remain hardware agnostic and focus on just creating good experiences without worrying about the underlying GPU architecture. Down to the simplest form, the graphics pipeline can be broken down into three main stages - Application, Geometry and Rasterization.

SSH- APPLICATION

The first and foremost stage is wherein the graphical data is yet to be calculated at all. If you take the example of playing a 3D game, then you'll no doubt have to interact with items in the video game world. You might want to pick up a brick and check out all the details of the brick. The mere act of reaching out to a brick with your virtual limbs, coming in contact with the brick and then

[JARGON BUSTER]

Core

The core is the component of the CPU that performs the actual processing. At times, all the cores can serve the same purpose, but in asymmetric designs, certain cores are specialised.

Counter

Hardware performance counters are specialised registers in microprocessors that count hardware related activities.

Data Line

The data line is an electrical line that ferries the data and determines if a bit is a 0 or 1.

DSP

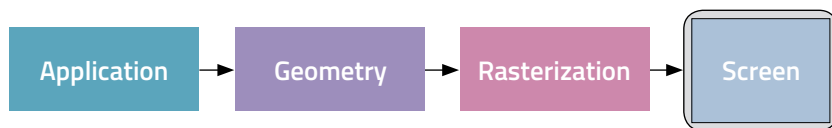
The DSP is a specialised processor for performing the complex mathematical calculations needed to work on digital signals.

made contact with the ball by constantly running collision detection algorithms. All the input and the interaction with the game world happens in the application stage. Once the application stage calculations are performed, the game passes on information such as where the

taking a visually identifiable form. The data, which is received at the geometry stage in the form of vertices and polygons has to be pieced together to form shapes and objects. In the graphics world, a vertex is a point in 3D space and a triangle is a plane formed by three vertices. Using a coordinate system, the positions of all the vertices and triangles are calculation to form objects. The same coordinate system can also be used to place objects relative to each other. Think of a room with a ball in the centre. Everything is generated using vertices and triangles. The room is built and the ball is then positioned relative to the room.

All the objects in the world that's being rendered in front of you isn't static. In fact, if you move around, the objects have to move relatively so that the position is maintained. The perspective from which you view the world is called the camera. And all the calculations performed thus far, have to be converted into another coordinate system where the camera is the origin. All the 3D objects that were calculated previously need to change based on from where you are looking at them. A cube appears like a square when viewed straight on. However, if you move slightly to the left or the right, the square becomes a cube and another dimension becomes visible to you. Such transformations have to be performed whenever the camera moves.

Now you have the various objects in the view port mapped but they aren't easily distinguishable from each other in terms of textures so the shaders come into play. The shaders can modify the texture, hue, saturation, brightness, contrast of every pixel on the screen. Since each scene has a light source or sources that are dynamic, the intensity of the light and the shadows projected by each object to be calculated by the shaders as well. The end result is a 3D scene with all the objects properly rendered, textured and lighting calculated.



Basic graphics pipeline

clasping your fingers around the brick are all calculations performed at the application level. As you were reaching out to the brick, the game had to keep on calculating when you hand actually made contact with the brick. This was done with the help of a technique called collision detection. In another example, if you were playing cricket and you hit the ball, the game decides that your bat

objects are and how do they appear to the next stage of the graphics pipeline, called geometry.

SSH- GEOMETRY

Since the volume of operations performed by the GPU are ridiculous. It's prudent to keep the information in its simplest state. Geometry is where the data from the application stage ends up

SSH- RASTERIZATION

The final stage is to rasterize the scene. You can think of rasterizing as taking a photograph form where the camera views the scene. The 3D scene is simply converted into a 2D image which forms one frame. Which is then sent to the